Stages 4–5: Visualization, Variable Injection, and Reproducibility I

Stage 4: Visualization

Stage 4 renders 16 publication-ready figures from the analysis outputs of Stages 2 and 3. All figures use the Wong (2011) colorblind-safe palette [wong2011colorblind] and enforce a 16-point minimum font size for accessibility compliance. Figures span six categories: field summary and domain distribution (2 figures), growth and temporal dynamics (2 figures), citation network topology (2 figures), hypothesis evidence dashboard and timeline (2 figures), assertion composition (2 figures), and text analytics—word cloud, PCA embeddings, term heatmap, dendrogram, topic-term bars, and co-occurrence matrix (6 figures). The figure generation script reads only JSON outputs and produces only PNG files, enforcing a strict, unidirectional data flow that prevents visualization operations from inadvertently modifying analytical results.

Stages 4–5: Visualization, Variable Injection, and Reproducibility II

Stage 5: Manuscript Variable Injection

Stage 5 computes dynamic variables from all pipeline outputs and injects them into manuscript Markdown templates via double-brace placeholder substitution of the form {<>} wrapping a variable name (e.g. the literal token spelled {{<CORPUS_SIZE>}} becomes the rendered corpus count). Variables include corpus-level metrics (size, year range, CAGR), per-domain counts and percentages, citation network statistics (nodes, edges, density, components, resolution rate, mean in-degree), hypothesis scores, and figure counts. All formatting (comma thousand separators, escaping) is applied during variable computation, ensuring the manuscript templates remain human-readable while producing publication-ready output. Unrecognized placeholders are preserved with a warning logged, enabling incremental manuscript development ahead of full pipeline execution.

Stages 4–5: Visualization, Variable Injection, and Reproducibility III

Reproducibility and Test-Driven Validation

The pipeline is deterministic given fixed random seeds and API responses. Test-driven development enforces 90% minimum code coverage on project modules and 60% on shared infrastructure, with real data and computation (no mocking). The test suite validates boundary conditions for hypothesis scoring (all-support $\rightarrow +1$, all-contradict $\rightarrow -1$, balanced $\rightarrow 0$), schema consistency, serialization round-trips, and end-to-end pipeline integrity. Source code, configuration, and outputs are available under CC-BY-4.0.